GUC
German University in Cairo

Lecture 10 Part 2

Trie Data Structure

CSEN 1038 Advanced Data Structures and Algorithms

Table of contents

Motivating Problem

Problem Statement: String Membership Query

The Task:

- **Input:** A predefined dictionary of N strings, $D = \{s_1, s_2, \ldots, s_n\}$.
- **Query:** A target string Q of length L.
- **Goal:** Return `True` if $Q \in D$, otherwise return `False`.

Example:

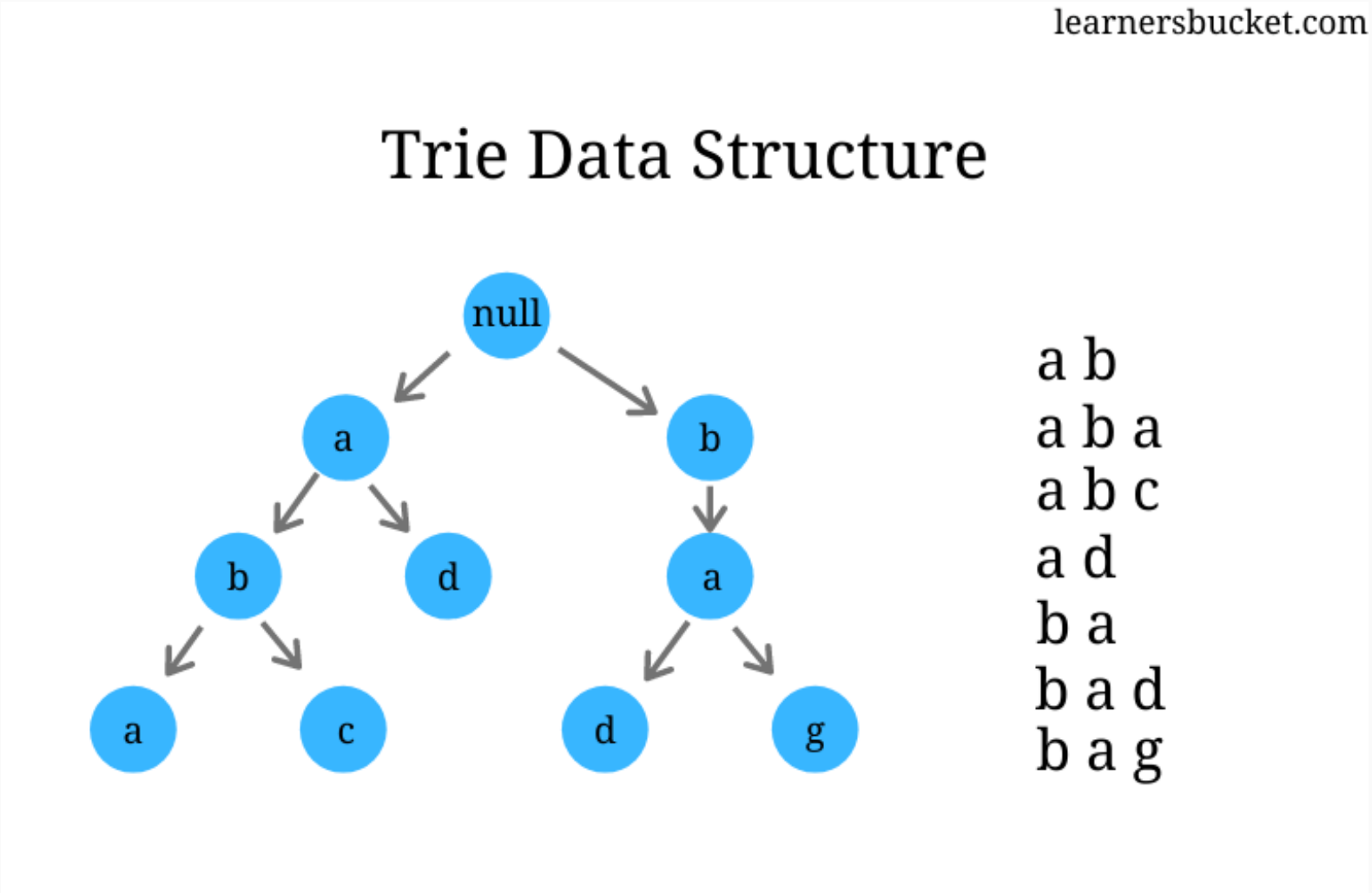
- $D = \{\text{"apple"}, \text{"apply"}, \text{"ball"}\}$
- Query: `"apple"` $\rightarrow$ **True**
- Query: `"app"` $\rightarrow$ **False**

Motivation: Standard search takes $O(N \cdot L)$. We want to achieve $O(L)$ by taking advantage of shared prefixes between strings.

What is a Trie?

What is a Trie?

A Trie (pronounced "try"), also known as a prefix tree, is a specialized tree-based data structure used to efficiently store and retrieve a collection of strings.



Trie Modifications

Trie Modifications

- Prefix & Suffix Searching: Beyond exact matches, Tries natively support prefix lookups. By inserting reversed versions of strings into a second Trie, you can efficiently support suffix queries as well.
- Dynamic Management: Unlike static arrays, a Trie can be modified in real-time. It supports both Addition (inserting new strings) and Deletion (removing strings by unsetting word markers or pruning unused nodes).
- Compressed Tries (Radix Tree): To save memory, nodes with only one child can be merged with their parent. This reduces the number of nodes and the overall depth of the tree.
- Frequency Tracking: By storing a "count" variable at each node, the Trie can instantly report how many words in the dataset share a specific prefix in $O(L)$ time.

(Optional) Trie on Integers

Integers as binary strings

While typically used for text, Tries can also be constructed using the binary representations of numbers (0s and 1s). This variation—often called a Bitwise Trie—is a surprisingly powerful tool for optimizing complex bitwise operations. Most notably, it provides an elegant and highly efficient solution to the classic algorithmic problem of finding the maximum XOR of two integers in an array, transforming what would normally be a slow, brute-force comparison into a lightning-fast path search down the tree.

Questions?

References i

- https://cp-algorithms.com/string/aho_corasick.html (Check the "Construction of the trie" section only)
- https://www.youtube.com/watch?v=e_0A_kw0sQQ&t=1s (Check only the "trie" part)